Class Activity 3 — JSONB and Vector Databases

DBAD 4000 · Group 5: Noah Jobse, Melody Shih, Altesse Imena

One PostgreSQL 16 database for a media-streaming service that stores data three ways in the same schema: **structured columns** for the fields we join and filter on, **JSONB** for data whose shape varies per row, and **vectors** (pgvector) for searching titles by meaning.

1. Design — why each kind of storage

- **Structured columns** for anything we join, filter, group, or constrain on: `users.plan`, `users.country`, `activity_log.event_type`, and the `user_id` / `title_id` foreign keys. These get real types and constraints so the database enforces them.
- **JSONB** for data whose shape legitimately varies: `users.preferences` (a `favourite_genres` array plus a subtitle language and a parental-controls flag) and `activity_log.details`, where a `play` event records `minutes_watched` and `device`, a `rate` event records `stars`, a `pause` event records `at_minute`, and a `search` event records a `query` and a result count.
- **Vectors** (`titles.embedding`, a 384-dim `vector`) for searching title descriptions by meaning rather than exact keywords.

The activity events use the **hybrid** pattern: the common, queryable fields (`user_id`, `title_id`, `event_type`, `occurred_at`) are real columns so we keep foreign keys and type checks, and the per-type fields live in one `details jsonb` column. The GIN indexes use `jsonb_path_ops` because our only JSONB filter is `@>` containment, which that operator class serves with a smaller, faster index.

Flexibility and scale. JSONB lets the schema absorb change without migrations: a new preference or a new event type with different fields just lands in the `preferences` / `details` document, so we never run `ALTER TABLE` or add nullable columns every product change, and one `activity_log` table holds every event shape instead of one table per type. It still scales because the GIN index makes `@>` containment lookups index-backed rather than full scans, so filtering 120 events, or millions, stays fast. Vector search scales the catalogue the same way: embeddings turn "find titles that mean something like this" into a nearest-neighbour distance, and the HNSW index answers those top-k queries in roughly logarithmic time, so semantic search over 60 titles or 60 million runs without comparing every row. Together they keep the parts that must be reliable (users, plans, foreign keys) in typed columns, while the parts that vary or grow (preferences, event details, meaning-based search) use structures built to flex and index at size.

2. Schema

Table	Key columns	JSON / vector
`users`	`user_id` PK, `name`, `email`, `country`, `plan`	`preferences jsonb`
`titles`	`title_id` PK, `title`, `kind`, `release_year`, `genre`, `description`	`embedding vector(384)`

Table	Key columns	JSON / vector
`activity_log`	`event_id` PK, `user_id` FK, `title_id` FK (nullable), `event_type`, `occurred_at`	`details jsonb`
`search_phrases`	`phrase_id` PK, `phrase`	`embedding vector(384)`

GIN index (`jsonb_path_ops`) on each JSONB column; HNSW index (`vector_cosine_ops`) on `titles.embedding`.

3. Data

- **Titles:** 60 rows, instructor-provided (`data/ca3_titles.csv`).
- **Users (24) and activity events (120):** synthetic, generated by seeded SQL (`setseed` makes them reproducible). Four event types: `play`, `pause`, `rate`, `search`.
- **Embeddings:** the 60 title descriptions and 7 search phrases were embedded with `all-MiniLM-L6-v2` (384 dimensions) in the provided notebook and upserted into the rows.

4. JSONB queries and results

Q1 — Containment (`@>`), served by the GIN index. Which users count "Mystery" among their favourite genres?

```
SELECT user_id, name, preferences->'favourite_genres' AS favourite_genres
FROM users
WHERE preferences @> '{"favourite_genres": ["Mystery"]}'
ORDER BY user_id;
```

> 7 users: 2, 6, 9, 13, 16, 20, 23.

Q2 — Extract (`->>`) + aggregate. Total minutes watched per title (play events only), top 10.

```
SELECT a.title_id, t.title,
       SUM((a.details->>'minutes_watched')::int) AS total_minutes,
       COUNT(*) AS play_events
FROM activity_log a JOIN titles t ON t.title_id = a.title_id
WHERE a.event_type = 'play'
GROUP BY a.title_id, t.title
ORDER BY total_minutes DESC LIMIT 10;
```

> Seven Summers 308 (4 plays), Byte Club 273 (3), Sweep the Nation 236 (3), Beneath the Dunes 217 (4), Night Rounds 176 (3), Fourth Quarter 163 (3), Neon Drift 154 (2), The Clockmaker's Apprentice 134 (2),

> Crumb & Clue 121 (2), Zero Day Morning 113 (1).

Q3 — `jsonb_array_elements_text`: unnest a JSON array. Which favourite genres are most common?

```
SELECT genre, COUNT(*) AS fans
FROM users, jsonb_array_elements_text(preferences->'favourite_genres') AS genre
GROUP BY genre ORDER BY fans DESC;
```

> Mystery 7, Thriller 7, Documentary 7, Comedy 7, Reality 7, Sci-fi 7, Drama 6.

5. Vector queries and results

All three use pgvector's cosine-distance operator `<=>` (0 = same meaning), served by the HNSW index on `titles.embedding`.

Q4 — Search by meaning. A viewer types "something scary in the woods"; the phrase was embedded into `search_phrases`. Which 5 titles mean the closest thing?

> The Woods Remember (Horror) 0.602, After Dark (Documentary) 0.638, Little Fox Detectives (Animation) 0.648, The Missing Hour (Mystery) 0.666, Wild Kitchen 0.671.

Q5 — "More like this". A viewer finished *Hollow Pines*; which 5 other titles are closest in meaning (itself excluded)?

> The Missing Hour 0.687, The Woods Remember 0.695, Glacier Station 0.704, Seven Summers 0.706, Wild Kitchen 0.714.

Q6 — Vector search combined with a filter from our other data. For user 16, recommend 5 titles similar to the last title they played (*Neon Drift*), excluding everything they have already played.

> Signal Lost (Thriller) 0.612, Office of Lost Things (Comedy) 0.711, The Missing Hour (Mystery) 0.716, Glacier Station (Mystery) 0.762, Midnight Ledger (Thriller) 0.764.

6. How to run

Needs Docker (or local PostgreSQL 16 with pgvector). From the repo root, run `sql/01` – `04`, then `02`'s `\copy` load, then the vector half `sql/06` – `07`, then the query files `sql/05` and `sql/08`. Full commands are in `README.md`. Verified end to end: 24 users, 60 titles all with 384-dim embeddings, 120 events, 7 search phrases; all six queries return the results above.

7. Contributions

Member	Work
Noah Jobse	Schema design and storage-model decisions; `users` / `titles` / `activity_log` DDL; seeded synthetic users and activity; the 3 JSONB queries; JSONB GIN index strategy; pre-declared the pgvector extension and `titles.embedding` so the two halves compose; README and schema diagram.
Altesse Imena	Ran the embeddings notebook (`all-MiniLM-L6-v2` , 384-dim); upserted title embeddings and built the `search_phrases` table; HNSW cosine index; the 3 vector queries.
Melody Shih	This report and the contribution table.

8. AI assistance

Per the activity's integrity note: the team designed the schema and chose the storage model, the column-versus-JSONB split, and the index strategy. AI (Claude) was used to generate the synthetic sample data and to write the SQL to the team's design decisions, and all of it was reviewed and run by the team. The title catalogue and the embeddings notebook are instructor-provided.